

Tether

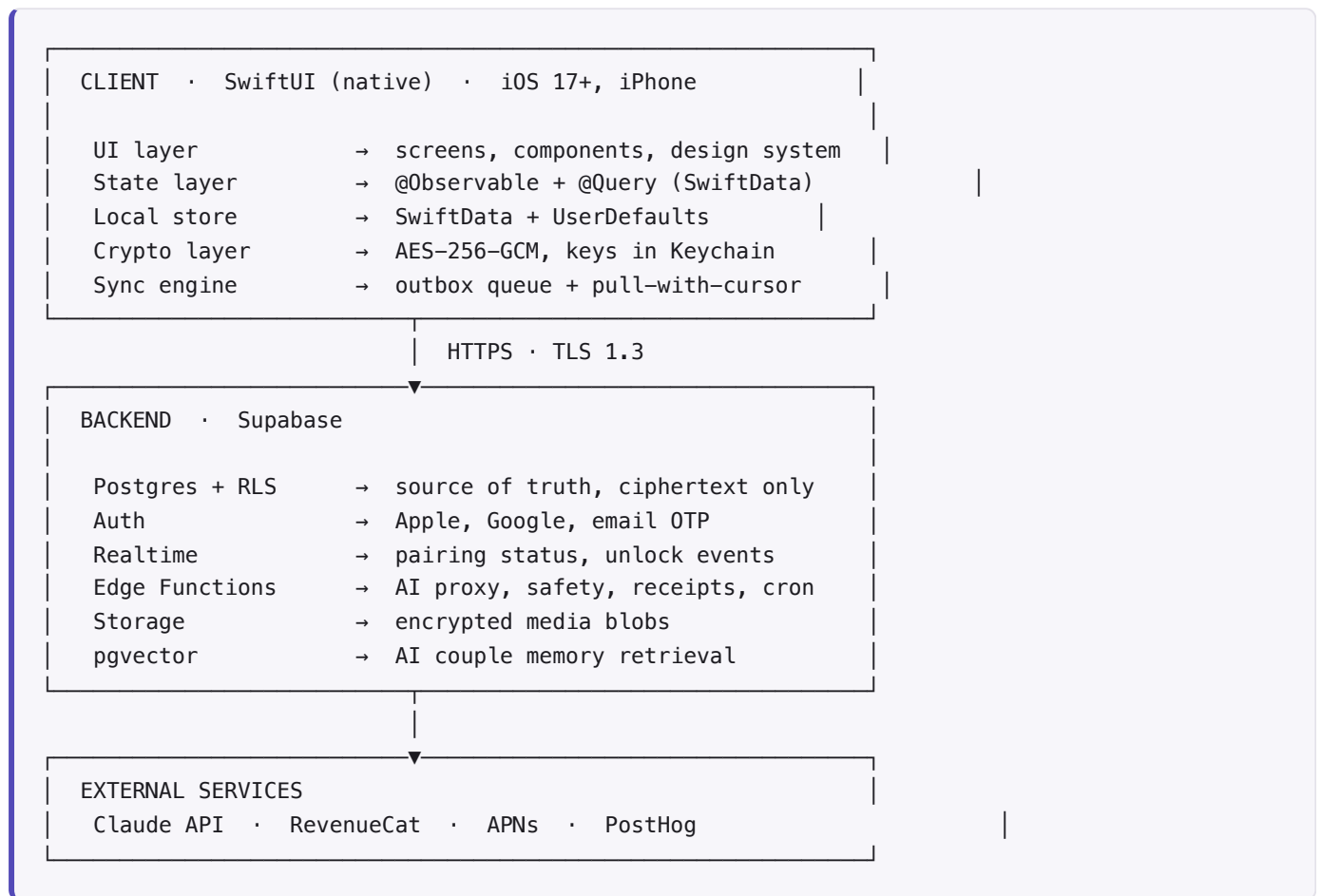
Architecture, stack, and engineering requirements for v0.1

Version 1.0 **Date** 17 Sept 2026 **Status** Approved for build **Platforms** iOS 17+ (iPhone)
Companion PRD · UI/UX · Backend Schema

Contents

- | | |
|-------------------------------------|-----------------------------------|
| 1. Architecture overview | 9. Push notifications |
| 2. Technology stack and rationale | 10. Payments |
| 3. Client architecture | 11. Analytics and instrumentation |
| 4. Offline-first sync strategy | 12. Security and encryption |
| 5. Backend architecture | 13. Performance and scalability |
| 6. API specification | 14. Third-party services and cost |
| 7. Authentication and authorisation | 15. Environments, CI/CD, testing |
| 8. AI pipeline | 16. Compliance |

01 Architecture overview



Architectural principle

Local-first. Every read is served from SQLite. The network is used only to sync. The app is fully usable on a plane, in a subway, or with a dead backend. This single decision eliminates an entire class of the sync bugs that affected the predecessor product.

02 Technology stack and rationale

Layer	Choice	Rationale
Framework	SwiftUI, native	iPhone-first per the build decision. Native gives the smoothest animation and the tightest Keychain and CryptoKit integration, which the privacy promise depends on. Trade-off accepted: Android is deferred, and roughly half the US market is therefore unaddressed until a separate build exists.
Language	Swift 5.9, strict concurrency	Non-negotiable for a codebase with encryption, offline sync, and money. Catches the class of bug you cannot afford to ship.
Local DB	SwiftData	Apple-native, offline by default, no dependency to maintain. Encryption is applied at the service boundary so sealed fields stay sealed at rest.
Key-value	MMKV	Synchronous, encrypted-at-rest, for settings and session state.
Server state	@Query (SwiftData)	Observation-driven reads. No network layer exists yet; the app is fully usable offline.
UI state	@Observable	Swift 5.9 Observation. No Combine, no ObservableObject boilerplate.
Backend	Supabase	Postgres with real relational couple modelling, Row Level Security, realtime, and edge functions. Cheap to start, portable — it is just Postgres.
Vector store	pgvector (in Supabase)	Keeps AI couple memory next to the data it references. No separate vector database to operate.
AI	Claude API	Strong instruction-following for a narrow safety-constrained domain, long context for couple memory, voice-ready.
Payments	RevenueCat (stubbed)	Handles StoreKit and Play Billing, receipt validation, cross-platform entitlement, and the "one sub covers both partners" sharing model.
Analytics	PostHog (self-hostable)	Product analytics without content leakage. Can be run on your own infrastructure, which supports the privacy promise.
Push	UNUserNotificationCenter → APNs	Local notifications shipped and working. Remote push is added alongside when the backend lands; identifiers stay the same.
Error tracking	Sentry	Scrub all content before send. Configure PII scrubbing explicitly.

03 Client architecture

```
Tether/  
  TetherApp.swift  app entry + RootView  
  components/      design system + feature components  
  features/  
    onboarding/    track selection, love languages, first prompt  
    pairing/        invite, waiting room, QR, merge ceremony  
    daily/          prompt, mood, reply, unlock  
    journal/        entries, export  
    pulse/          relationship state, weekly recap  
    coach/          AI conversation, voice  
    solo/           solo-mode surfaces  
    paywall/        pricing, trial, restore  
  services/  
    crypto/         key management, encrypt/decrypt  
    sync/           outbox, pull, conflict resolution  
    ai/             client wrapper, streaming  
    notifications/  scheduling, send-time learning  
    purchases/      RevenueCat wrapper  
  db/              WatermelonDB schema, models, migrations  
  lib/            utils, constants, feature flags
```

Key client rules

- **No component talks to the network directly.** Everything goes through the sync engine or a service layer. This keeps offline behaviour consistent by default.
- **Encryption happens at the service boundary, never in the UI.** Components handle plaintext; the crypto service handles everything else.
- **Optimistic writes always.** The UI updates instantly; sync confirms later. Users never wait on a spinner for their own action.
- **Feature flags from day one.** Remote-configurable so you can kill the AI coach or a content track without a store release.

04 Offline-first sync strategy

Write path

- 1 **User acts** — writes a reply, taps a mood.
- 2 **Encrypt locally** — content encrypted on device.
- 3 **Write to SQLite** — committed to local store immediately.
- 4 **Enqueue in outbox** — durable queue, survives app kill.
- 5 **Flush** — when connectivity allows, batches of encrypted records.
- 6 **Ack and clear** — server confirms, outbox entry removed.

Read path

- 1 **Pull with cursor** — client sends last-synced cursor, receives only deltas.

- 2 **Decrypt and merge** — into local SQLite.
- 3 **Emit to UI** — observables refresh automatically.

Conflict resolution

Data type	Strategy	Rationale
Journal entries	Last-write-wins on <code>updated_at</code> , keep both versions	Writes are per-user and immutable in practice; collisions are near-impossible.
Mood logs	Last-write-wins per user per day	Naturally keyed; no merge needed.
Profile / settings	Last-write-wins with monotonic version	Low-stakes, easily re-set.
Pairing state	Server authoritative	Correctness matters more than latency. Never guess here.
Streaks	Server computed, client displays	Prevents cheating and divergence.

Idempotency

Every outbox record carries a client-generated UUID. The server treats it as an idempotency key. This makes retries safe, which is what prevents duplicate entries under flaky mobile networks.

05 Backend architecture

Supabase services used

- **Postgres** — source of truth. Stores ciphertext for journal and chat; plaintext only for non-sensitive relational data.
- **Row Level Security** — every table protected. A user can only ever read their own rows and their partner's shared rows.
- **Auth** — Apple, Google, email OTP. No phone number, deliberately.
- **Realtime** — pairing status, prompt unlock events, typing indicators. Used sparingly; most state arrives via pull.
- **Edge Functions (Deno)** — AI proxy, safety classification, webhook handlers, scheduled jobs.
- **Storage** — encrypted media only, private buckets.

Edge functions

Function	Responsibility
<code>ai-coach</code>	Safety-classify input → retrieve couple memory → build grounded prompt → stream response → store both sides encrypted.
<code>safety-scan</code>	Classify journal entries on write. On hit, suppress and return resource payload.
<code>prompts-daily</code>	Cron. Assign the day's prompt per couple, honouring both partners' tracks and time zones.
<code>notifications-send</code>	Cron. Per-user send-time optimisation, staggered partner delivery.
<code>pulse-compute</code>	Cron. Compute Relationship Pulse from mood, cadence, sentiment.
<code>recap-weekly</code>	Cron. Build and deliver the Sunday recap.
<code>webhook-revenuecat</code>	Sync entitlement to both partners. Handle renewals, refunds, grace periods.
<code>webhook-clerk-none</code>	—

06 API specification

Client talks to Supabase via PostgREST for CRUD, and to edge functions for anything involving AI, safety, or third-party secrets. All requests require a valid JWT.

Method	Endpoint	Purpose
POST	<code>/functions/v1/ai-coach</code>	Streaming AI response. Body: <code>{message, conversationId, mode}</code>
POST	<code>/functions/v1/safety-scan</code>	Classify journal text before commit
POST	<code>/functions/v1/pairing/create-invite</code>	Generate link, QR payload, and 6-digit code
POST	<code>/functions/v1/pairing/redeem</code>	Redeem a code or QR; creates the couple
POST	<code>/functions/v1/pairing/nudge</code>	Send nudge to pending invitee
GET	<code>/rest/v1/prompts?day=eq.</code>	Fetch daily prompt (pull)
POST	<code>/rest/v1/prompt_replies</code>	Submit encrypted reply
GET	<code>/rest/v1/sync?cursor=gt.</code>	Delta pull
POST	<code>/rest/v1/sync</code>	Batched outbox flush
POST	<code>/functions/v1/account/export</code>	Generate full data export
DELETE	<code>/functions/v1/account/delete</code>	Hard delete, 30-day grace, then purge
POST	<code>/functions/v1/webhook-revenuecat</code>	Entitlement sync

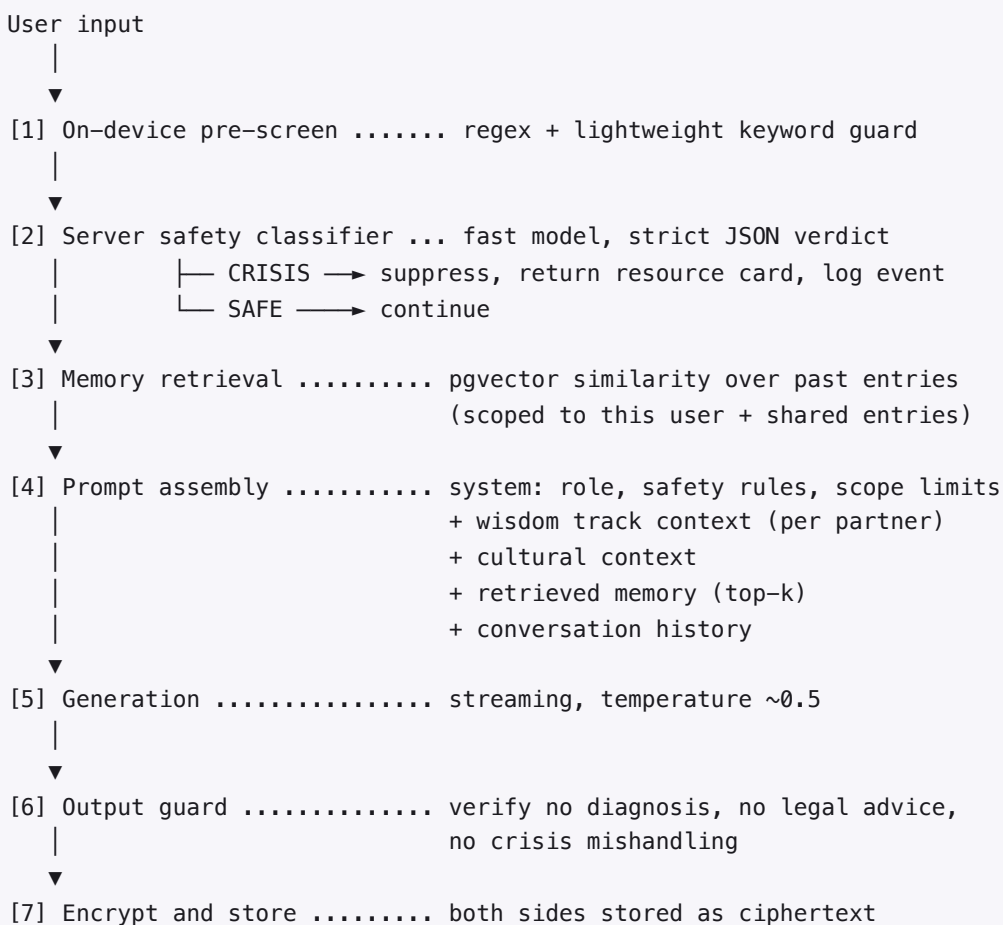
Versioning

Edge functions are versioned in the path (`/v1/`) from day one. Client ships a minimum-supported-version header; older clients get a graceful upgrade prompt rather than a crash.

07 Authentication and authorisation

- **Methods** — Sign in with Apple (required by App Store if any social login is offered), Google, email OTP.
No phone number — a competitor loses users at this step.
- **Sessions** — JWT with refresh rotation. Biometric re-auth optional for the private vault.
- **Row Level Security** — the real authorisation layer. Policies enforce: you see your rows; you see your partner's rows only where `visibility = 'shared'` and you are in an active couple.
- **Invite tokens** — single-use or 7-day expiry, revocable, bound to the inviting user.
- **Anonymous pairing** — a partner can redeem an invite before creating an account; state transfers on signup.

08 AI pipeline



Requirements

- **Latency** — first token under 3s at p95. Stream tokens, never block on full response.

- **Memory scope** — retrieval is scoped to the asking user plus shared entries. **Never retrieve the partner's private entries.**
- **Caching** — cache responses to common prompt patterns. Use a small fast model for classification, a large one for coaching.
- **Cost control** — per-tier monthly message caps, token budget alerts, response length limits.
- **Refusal behaviour** — must refuse gracefully and warmly, never lecture. A refusal that shames the user is a worse outcome than a slightly generic answer.

09 Push notifications

Notification	Trigger	Rules
Daily prompt	Daily, personalised time	Per-partner send time. Never both at the same minute.
Partner replied	On unlock	Highest-value notification. Never suppress unless paused.
Pairing nudge	24h, 72h	Max 3 total, then stop.
Streak at risk	Evening if unanswered	Offers freeze, not guilt. Max 1/day.
Weekly recap	Sunday	Opt-out available.
Re-engagement	Day 14, 30 dormant	Nostalgic, not discount-based.

Send-time optimisation — record each user's active hours over 14 days, then schedule within their window. Store per-user, recompute weekly. **Time-zone aware** for long-distance couples.

10 Payments

- **RevenueCat** wraps StoreKit 2 and Play Billing. Products: monthly, annual, Couples+ annual.
- **Shared entitlement** — on purchase, write entitlement to both `couple_members` rows. Either partner purchasing unlocks both.
- **Webhook** — handles renewal, refund, grace period, billing retry, and crossgrade.
- **Intro offers** — 14-day free trial configured in App Store Connect and Play Console, surfaced through RevenueCat.
- **Restore purchases** — prominent, one tap. Required by App Store review.
- **Cancellation** — handled by the stores, but the app links directly. No dark patterns.

11 Analytics and instrumentation

Hard rule

Never send journal content, AI message content, or any user text to analytics. Event names and properties only. This is a product promise, not a preference.

Core events

Event	Properties
onboarding_step_completed	step, duration_ms
track_selected	track_id
invite_sent	method (link/qr/code)
invite_redeemed	method, hours_since_sent
pairing_completed	method, elapsed_hours
solo_mode_entered	trigger (immediate/72h_fallback)
prompt_answered	day_index, has_voice, duration_s
streak_extended / streak_frozen	length
coach_message_sent	mode (text/voice), latency_ms
safety_event	category, action_taken (no content)
paywall_shown / trial_started / converted	placement, product

12 Security and encryption

Control	Implementation
Transport	TLS 1.3, certificate pinning on the AI edge function.
At rest, client	SQLite encrypted via SQLCipher. Keys in iOS Keychain / Android Keystore.
At rest, server	Journal entries and AI messages stored as AES-256-GCM ciphertext. Server holds no key.
Key derivation	Per-user key derived from an auth-bound secret, wrapped by device key. Documented recovery trade-off: loss of all devices means loss of encrypted history.
Access control	Row Level Security on every table. Tested with automated policy tests in CI.
Secrets	Edge function secrets only. No API keys in the client bundle. Verified in CI.
Logging	Sentry and server logs scrub content. Allowlist, not denylist.
Backup	Daily encrypted Postgres backups, 30-day retention, restore tested quarterly.

13 Performance and scalability

Metric	Target	Notes
Cold start	< 2.0s	p95, mid-range device
Prompt screen interactive	< 1.0s	Local read, no network
AI first token	< 3.0s	p95, streaming
Sync flush (50 records)	< 2.0s	On reasonable network
API availability	99.5%	Excludes planned maintenance
Crash-free sessions	> 99.5%	Release-blocking below this

Scaling path

- **0–10k users** — Supabase free/pro tier. No changes needed.
- **10k–100k** — read replicas for pull, connection pooling (PgBouncer), edge function regionalisation, AI response caching.
- **100k+** — partition `prompt_replies` and `journal_entries` by month, archive cold data, consider dedicated vector store if pgvector latency degrades.

14 Third-party services and cost

Service	Free tier	Est. at 10k MAU	Notes
Supabase	Yes	\$25–75/mo	Pro tier at launch for backups and support
Claude API	Pay as you go	\$250–700/mo	Largest variable cost. Caching and caps essential.
RevenueCat	Up to \$2.5k MTR	Free → \$0	Percentage only above threshold
Apple Developer Program	No	\$99/yr	Required for TestFlight and App Store
PostHog	1M events	\$0–45/mo	Self-hostable if privacy requires
Sentry	5k errors	\$0–26/mo	Configure scrubbing
Domain + misc	—	\$20/mo	tether.app, email, USPTO filing
Estimated total at 10k MAU		~\$350–950/mo	

15 Environments, CI/CD, and testing

Environments

- **local** — Supabase local via CLI, seeded fixtures.

- **staging** — separate Supabase project, sandbox RevenueCat, test AI keys.
- **production** — separate project, strict RLS, monitoring on.

CI pipeline

- 1 **Lint + typecheck** — ESLint, strict TS. Fails fast.
- 2 **Unit tests** — Vitest. Crypto, sync logic, streak math, Pulse computation.
- 3 **RLS policy tests** — automated verification that users cannot read others' data. **This is a release gate.**
- 4 **Safety red-team suite** — fixed corpus of crisis inputs; must classify correctly with zero misses.
- 5 **E2E tests** — Detox or Maestro on the critical paths: onboarding, pairing, daily loop, paywall.
- 6 **Build** — EAS Build, distribute to TestFlight / internal track.

Manual test matrix before release

- Airplane mode: write 10 entries, reconnect, verify all sync and none duplicate.
- Pairing: all three methods, including in-person QR with both devices offline.
- Interrupted purchase, restore purchases, cancellation.
- Dynamic Type at 200% on every screen.
- VoiceOver traversal of the daily loop.

16 Compliance

Requirement	Action
App Store	Account deletion in-app (required). Sign in with Apple if any social login. Privacy nutrition labels accurate. Subscription terms visible on paywall.
Play Store	Data safety form. Account deletion. Subscription disclosure.
CCPA / CPRA	Do-not-sell. Data export and deletion. Privacy policy published before launch.
GDPR	Lawful basis, data portability, erasure. Needed before EU expansion; build the capability now.
Health claims	Never claim to treat, diagnose, or prevent any condition. Avoid triggering FDA wellness-device scrutiny. Legal review of all marketing copy.
Telehealth boundary	Clear, persistent disclaimer: not therapy, not a crisis service. Documented in ToS.
Liability	LLC or corp formed before launch. E&O insurance considered once revenue exists.